

FastBPE: Benchmarking and Optimizing Tokenizer Speed

Research-style benchmark report

FastBPE benchmarks tokenizer throughput, latency, memory, batching behavior, and domain sensitivity across English, code, web, and technical text. In the current five-trial run on Windows-10-10.0.26200-SP0, tiktoken was the strongest average MB/s performer at 4.735 MB/s. SentencePiece remained competitive and led the code domain at 4.988 MB/s. The cached Python tokenizer improved on the naive Python baseline by 1.47x in MB/s with higher memory usage.

Hypotheses

- H1. Production native tokenizer libraries will outperform readable Python baselines on throughput and latency.
- H2. Tokenizer rankings will change across English prose, code, noisy web text, and technical markdown-like text.
- H3. Repeated-substring caching will improve the custom Python baseline while increasing memory usage.
- H4. Batch encoding will favor tokenizers that expose efficient multi-document paths, but speed gains will not be uniform.
- H5. Exact token-ID compatibility cannot be claimed unless the tokenizer intentionally matches the reference vocabulary and merge logic.

Methodology and Experimental Design

Environment. Platform: Windows-10-10.0.26200-SP0. Python: 3.11.9. Processor: Intel64 Family 6 Model 189 Stepping 1, GenuineIntel.

Tokenizers. tiktoken, hf, sentencepiece, naive, cached.

Domains. code, english, technical, web.

Controls. Warmup before timing, separate cold-start measurement, identical document sets per run, five trials, and raw CSV/JSON output.

Datasets. This run used persisted synthetic corpora in data/ rather than the in-memory homogeneous fallback. The corpus was diversified specifically to reduce misleading cache behavior from repeated identical documents.

Exactness. tiktoken is the reference tokenizer, but this run did not include a non-reference tokenizer that claimed token-ID compatibility, so exact-match tables are absent by design.

Experiments

Experiment 1: Overall tokenizer speed, measured in MB/s, tokens/s, documents/s, and latency percentiles.

Experiment 2: Domain sensitivity across clean English, code, noisy web text, and technical text.

Experiment 3: Input-length scaling across the available length buckets in the current corpus.

Experiment 4: Batch versus single encoding, reported for tokenizers that support `encode_batch`.

Experiment 5: Exactness testing, included as a framework feature but not exercised for a non-reference compatible tokenizer in this run.

Experiment 6: Caching optimization, comparing the naive and cached Python prototypes.

Overall Results by Tokenizer

This table aggregates mean performance across all domains and trials.				
tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	4.735	1118675	0.2269	16768.0
hf	3.454	866205	0.3065	17920.0
sentencepiece	3.294	1632086	0.3251	31781.2
cached	2.258	2044328	0.4977	116589.4
naive	1.539	1391675	0.7082	38027.2

Top Tokenizer-Domain Configurations

These are the strongest individual tokenizer-domain combinations by MB/s.					
tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	technical	5.542	1106087	0.2630	14575.2
tiktoken	code	4.988	1168626	0.4092	19938.4
tiktoken	english	4.784	1180900	0.1878	21719.2
hf	code	3.964	870806	0.5173	20040.0
hf	technical	3.908	838216	0.3739	16108.0
sentencepiece	code	3.784	1499303	0.5402	34436.8
tiktoken	web	3.628	1019087	0.0473	10839.2
sentencepiece	english	3.516	1699931	0.2555	36656.0
hf	english	3.392	902459	0.2685	22228.0
sentencepiece	technical	3.321	1692778	0.4383	33520.0

Domain Sensitivity: MB/s by Domain

Throughput shifts materially by domain, which changes the ranking of libraries.					
domain	cached	ht	naïve	sentencepiece	tiktoken
code	2.506	3.964	1.613	3.784	4.988
english	2.115	3.392	1.556	3.516	4.784
technical	2.596	3.908	1.708	3.321	5.542
web	1.815	2.553	1.278	2.555	3.628

Cold Start and Batch Results

Cold start is reported for all tokenizers. Batch metrics are available only for adapters with batch support.					
tokenizer	metric	value	docs_per_s	tokens_per_s	Batch_latency_ms
cached	cold_start_ms	0.0930			
naive	cold_start_ms	0.1197			
tiktoken	cold_start_ms	0.1274			
sentencepiece	cold_start_ms	0.2199			
hf	cold_start_ms	0.3659			
cached			26767.2	13764781	0.5642
naive			17004.5	8201045	0.9750
hf			10637.0	1953269	0.9953
sentencepiece			8699.9	3712990	1.0265

Input-Length Scaling Summary

The current persisted corpus produced two dominant length buckets, so the length-scaling analysis is informative, but not yet broad.

tokenizer	length_bucket	latency_ms	bytes	tokens
cached	1025-4096	0.7468	1850.4	1598.3
cached	257-1024	0.2757	589.2	484.6
cached	65-256	0.0827	140.5	129.2
hf	1025-4096	0.4594	1850.4	395.8
hf	257-1024	0.1665	589.2	146.1
hf	65-256	0.0549	140.5	48.4
naive	1025-4096	1.0764	1850.4	1598.3
naive	257-1024	0.3587	589.2	484.6
naive	65-256	0.1116	140.5	129.2
sentencepiece	1025-4096	0.4936	1850.4	791.2
sentencepiece	257-1024	0.1627	589.2	275.9
sentencepiece	65-256	0.0542	140.5	90.4
tiktoken	1025-4096	0.3425	1850.4	395.0
tiktoken	257-1024	0.1157	589.2	134.5
tiktoken	65-256	0.0406	140.5	40.4

Caching Optimization Results

Repeated-substring caching improved throughput and latency relative to the naive Python baseline, with a

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_byte	cache_hit_rate
cached	2.258	2044328	0.4977	116589.4	0.9582
naive	1.539	1391675	0.7082	38027.2	N/A

Results and Discussion

Headline findings

Overall winner by mean MB/s: tiktoken at 4.735 MB/s and 0.2269 ms average latency.

Fastest code-domain tokenizer: tiktoken at 4.988 MB/s.

Cached versus naive Python baseline: 1.539 to 2.258 MB/s, or 1.47x speedup, with peak memory rising from 38027.2 to 116589.4 bytes.

Cold start ranking by mean latency favored naive (0.0930 ms) and tiktoken (0.1274 ms), while hf was slowest at 0.3659 ms.

Batch throughput favored the cached baseline at 26767.2 docs/s, but tiktoken is absent from the batch table because the current adapter intentionally exposes single-document encoding only.

Discussion

The results support H1. Native tokenizer libraries clearly outperformed the Python baselines on MB/s and latency, with tiktoken and SentencePiece consistently ahead of naive Python code.

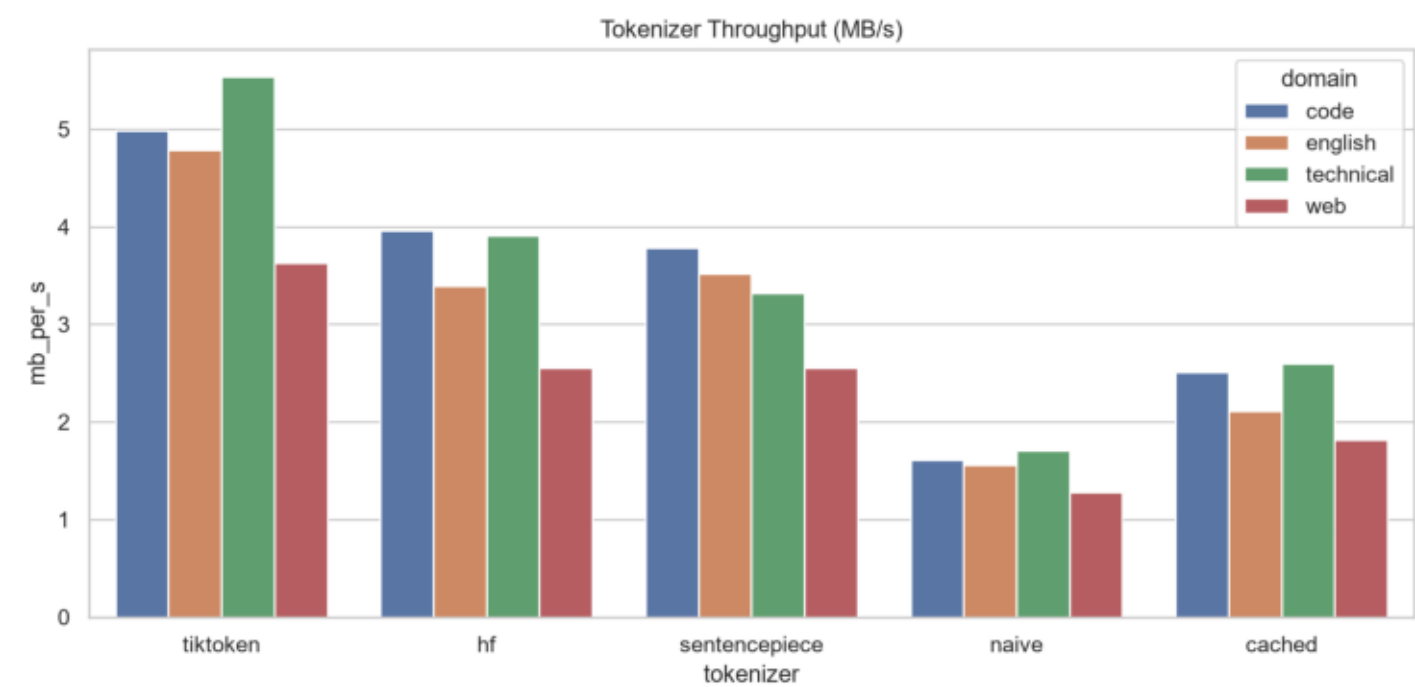
The results support H2. Ranking changed by domain: tiktoken led English, web, and technical text, while SentencePiece led code. That matters because tokenizer selection for RAG preprocessing, web corpora, and code corpora should not rely on one aggregate benchmark number.

The results support H3 with caveats. Caching materially improved the Python baseline, but it increased memory and still depended on high substring reuse. Even after diversifying the synthetic corpus, the cache hit rate remained high, so the next step is to rerun on real corpora.

The results partially support H4. Batch throughput was much higher for adapters with `encode_batch` support, but batch results are not comparable for tiktoken until a batch-capable adapter path is added.

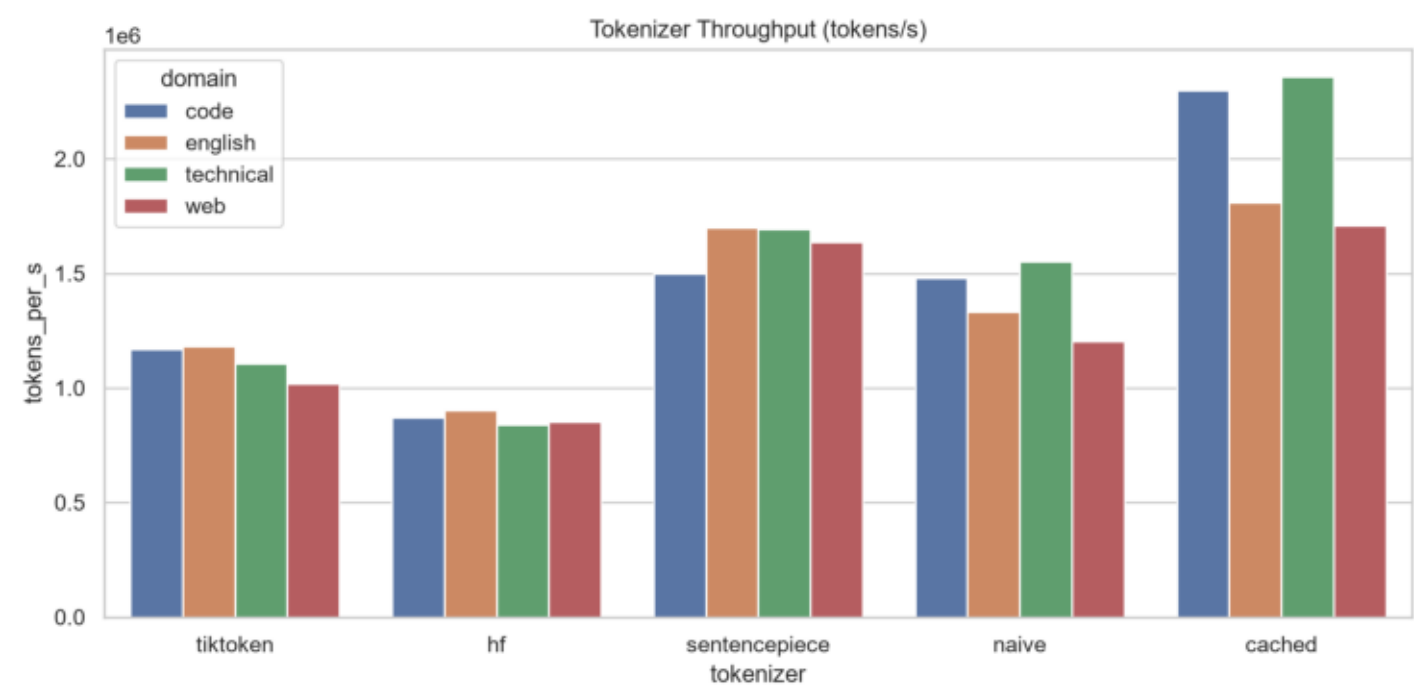
H5 remains unresolved empirically. The benchmark infrastructure is ready for exactness validation, but the project still needs a truly tiktoken-compatible optimized tokenizer before any claim about exact accelerated OpenAI-compatible BPE can be defended.

Throughput Mb S



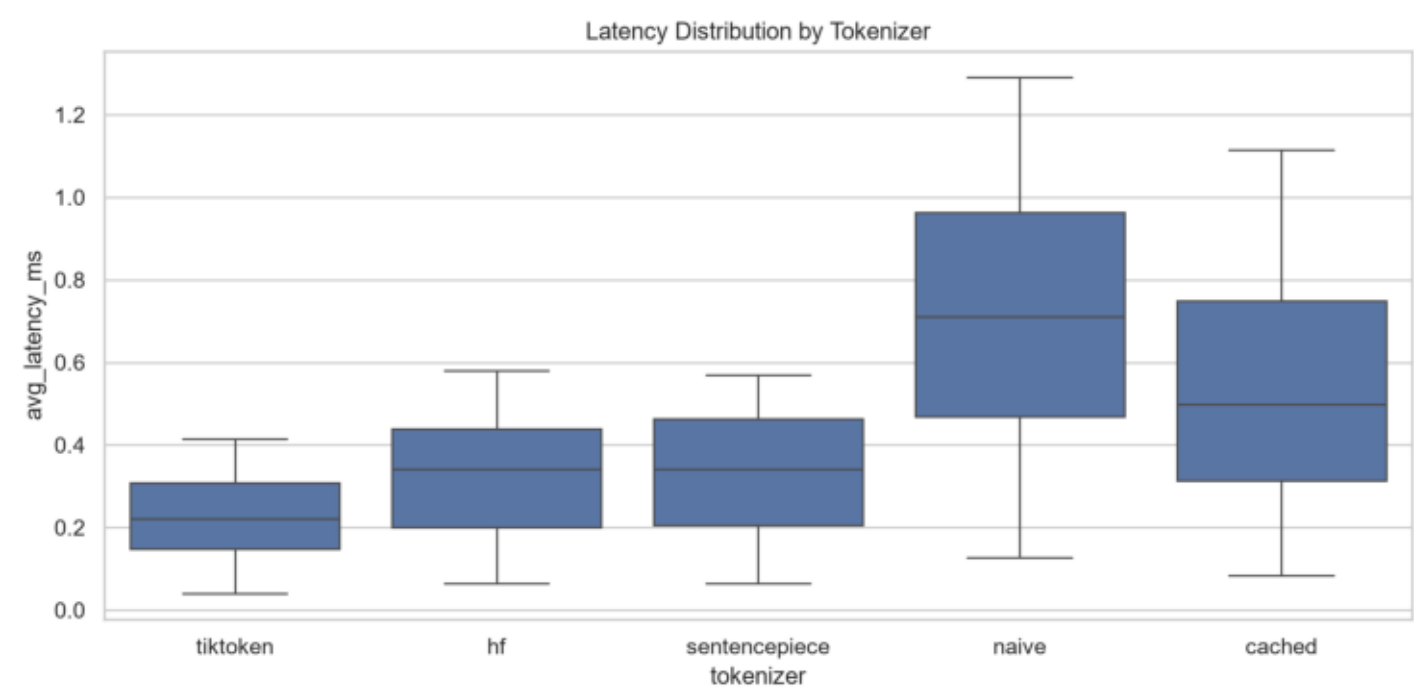
Experiment 1: Tokenizer throughput comparison in MB/s.

Throughput Tokens S



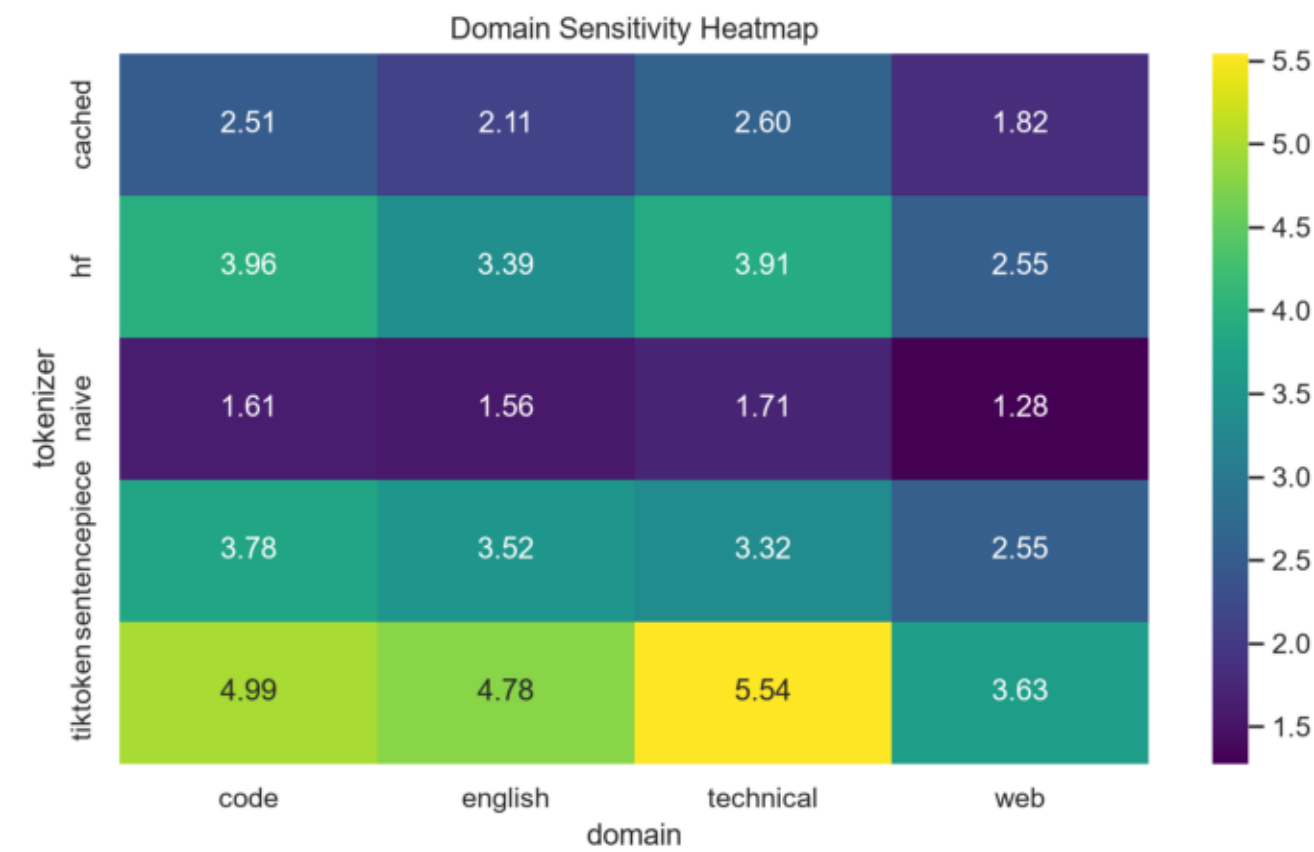
Experiment 1: Tokenizer throughput comparison in tokens/s.

Latency Distribution



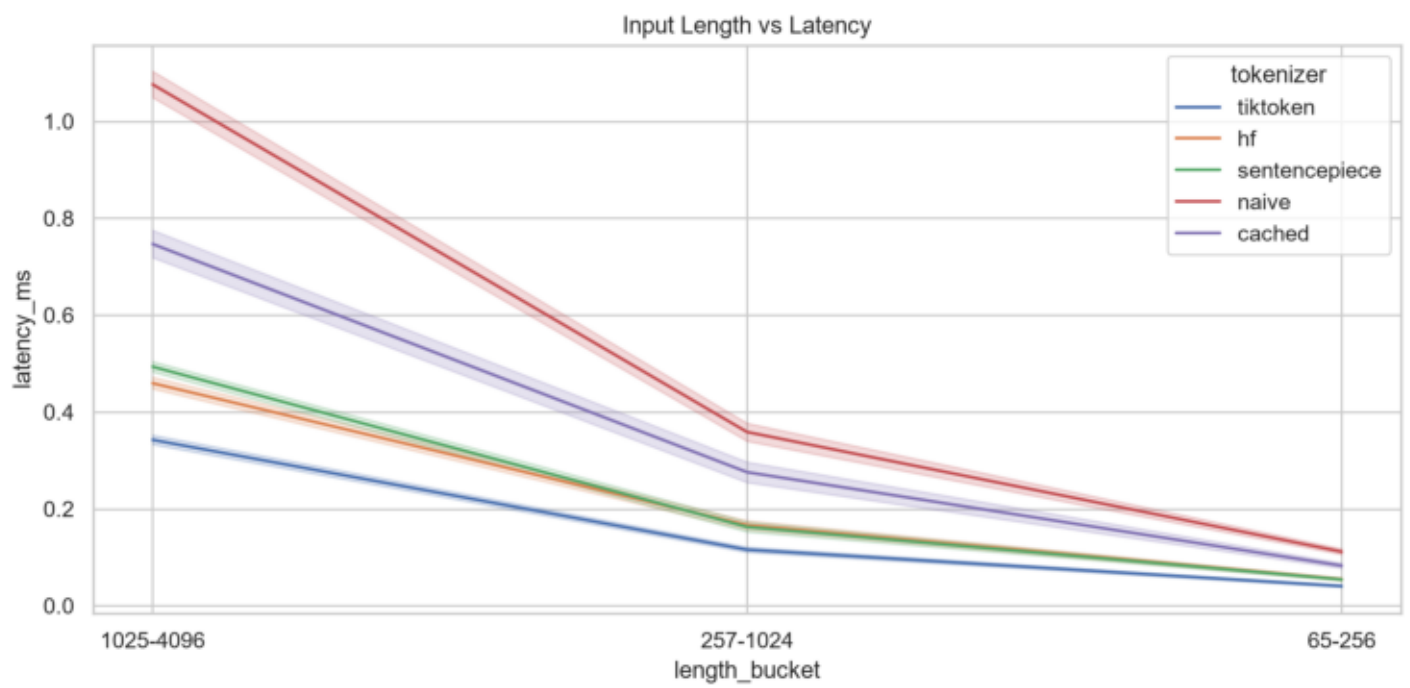
Latency distribution by tokenizer across benchmark trials.

Domain Heatmap



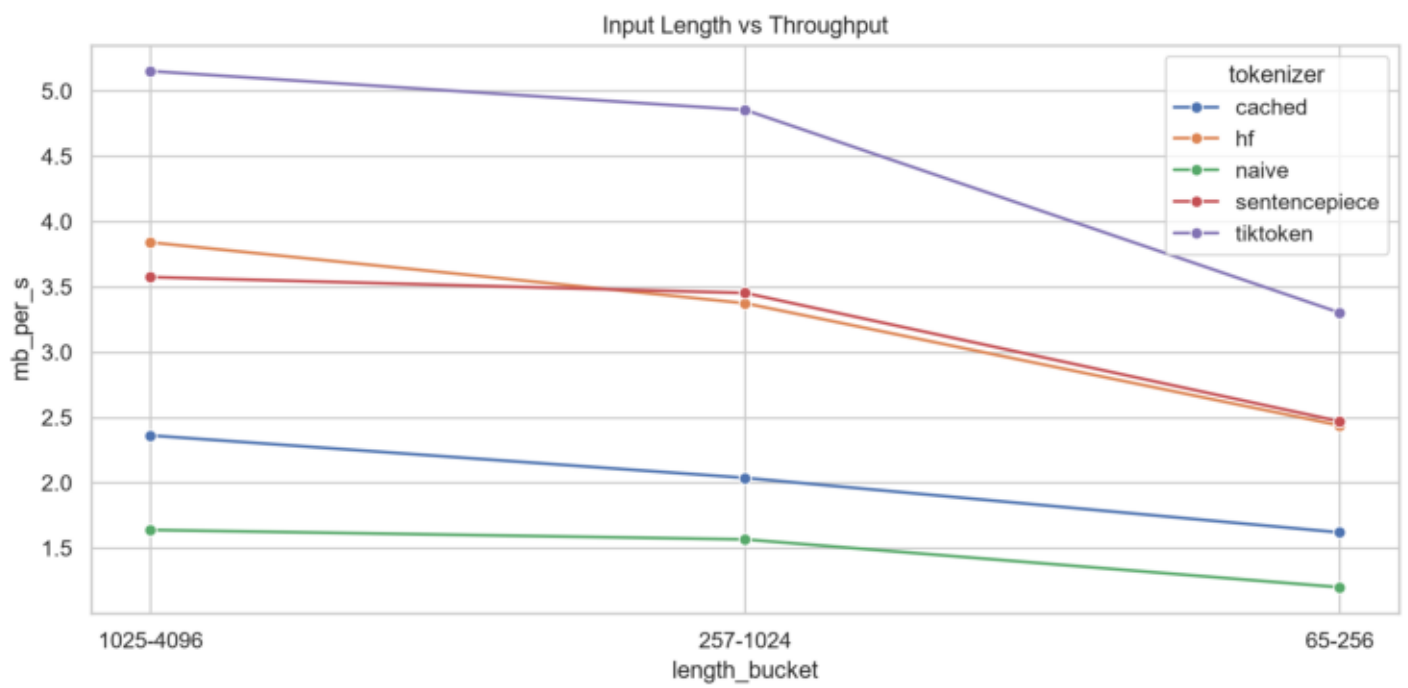
Experiment 2: domain sensitivity heatmap for throughput.

Length Vs Latency



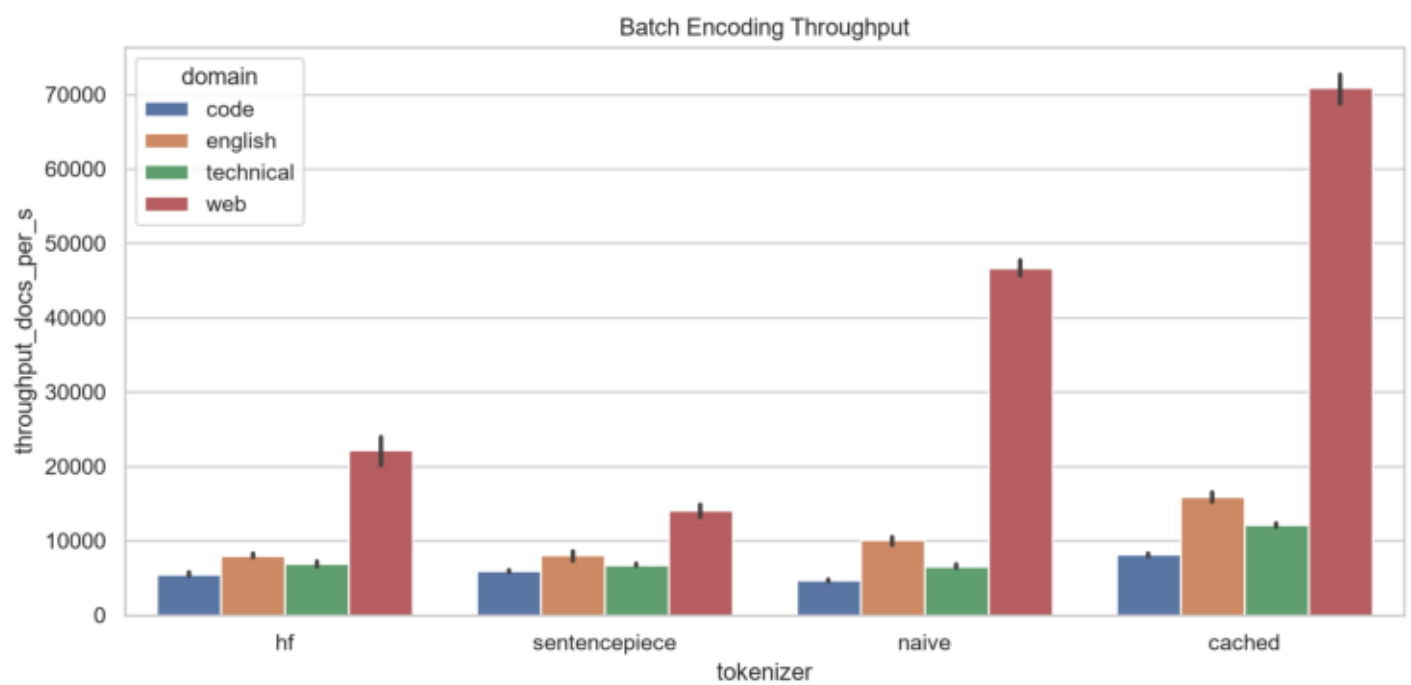
Experiment 3: input length versus latency.

Length Vs Throughput



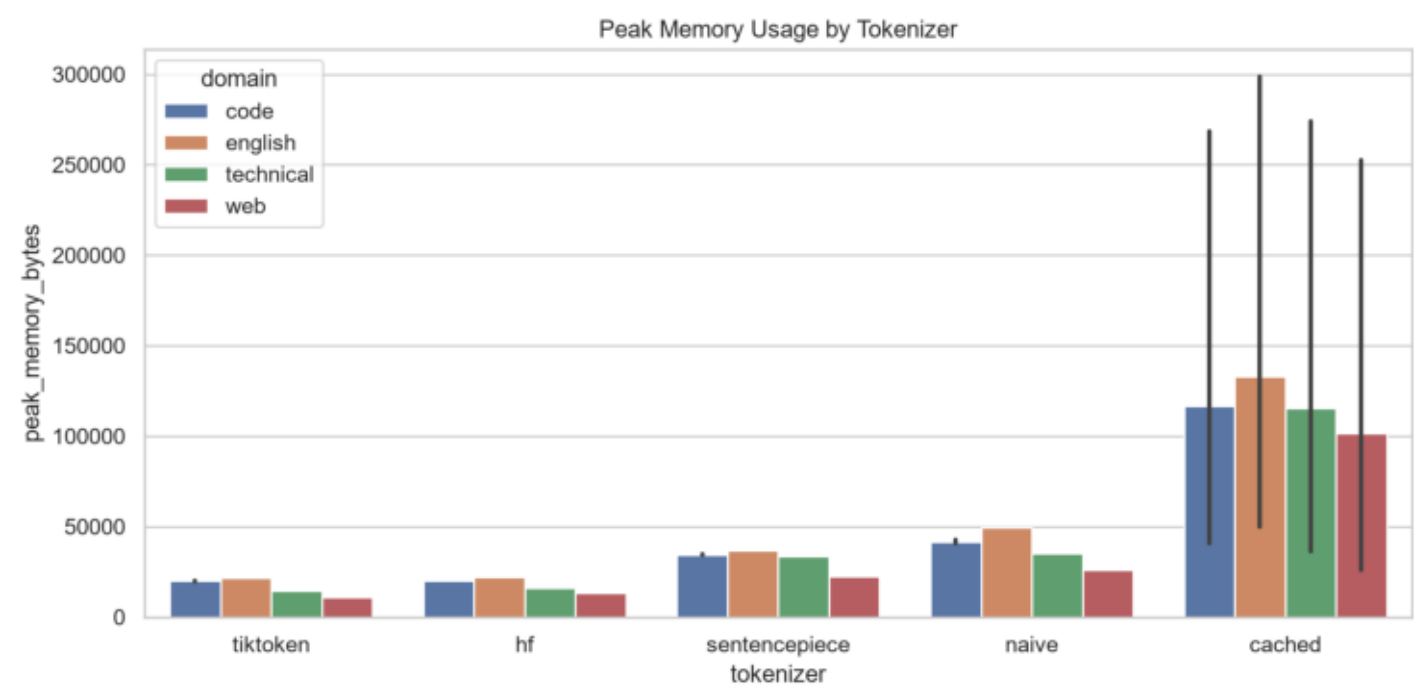
Experiment 3: input length versus throughput.

Batch Throughput



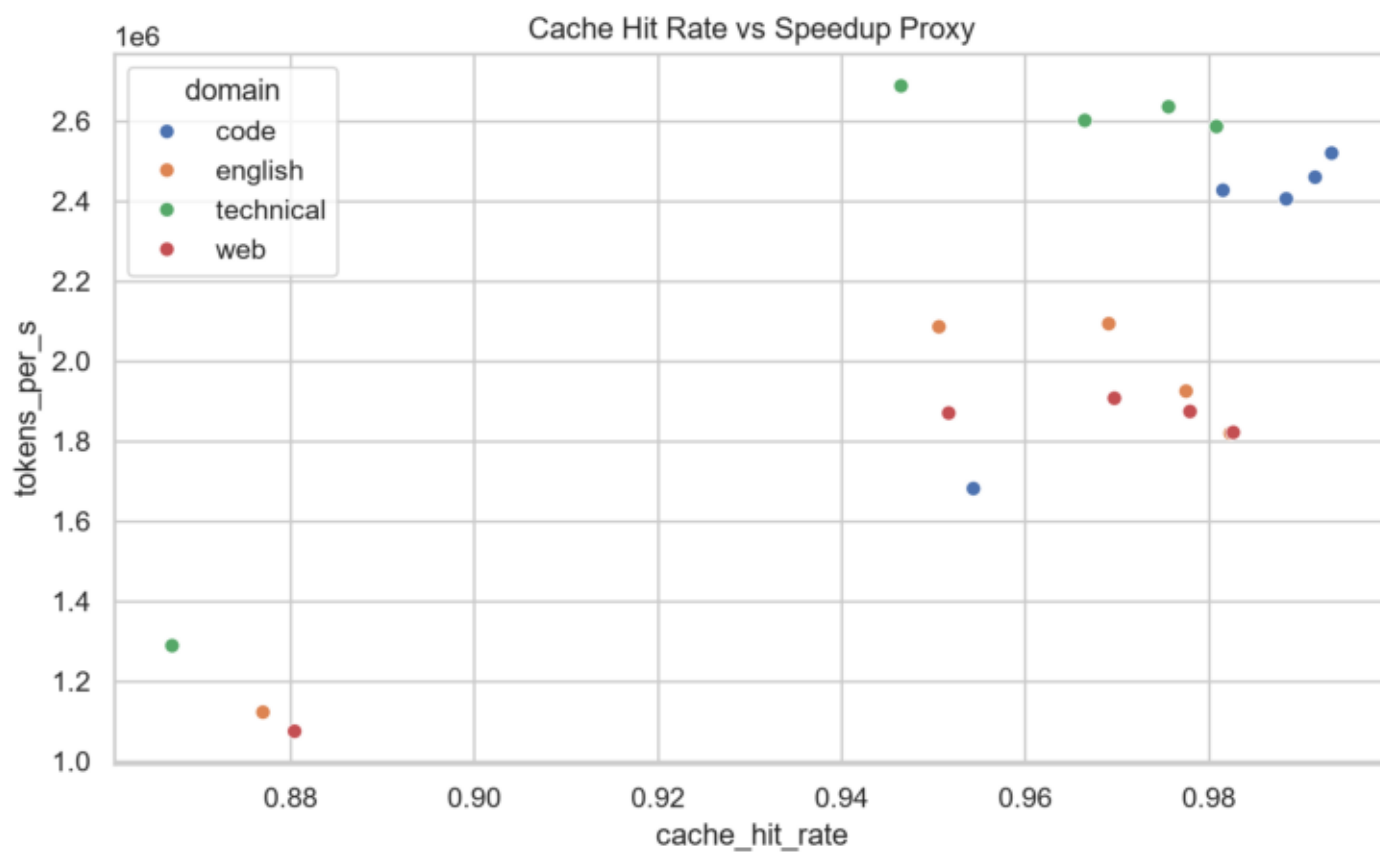
Experiment 4: batch encoding throughput.

Memory Usage



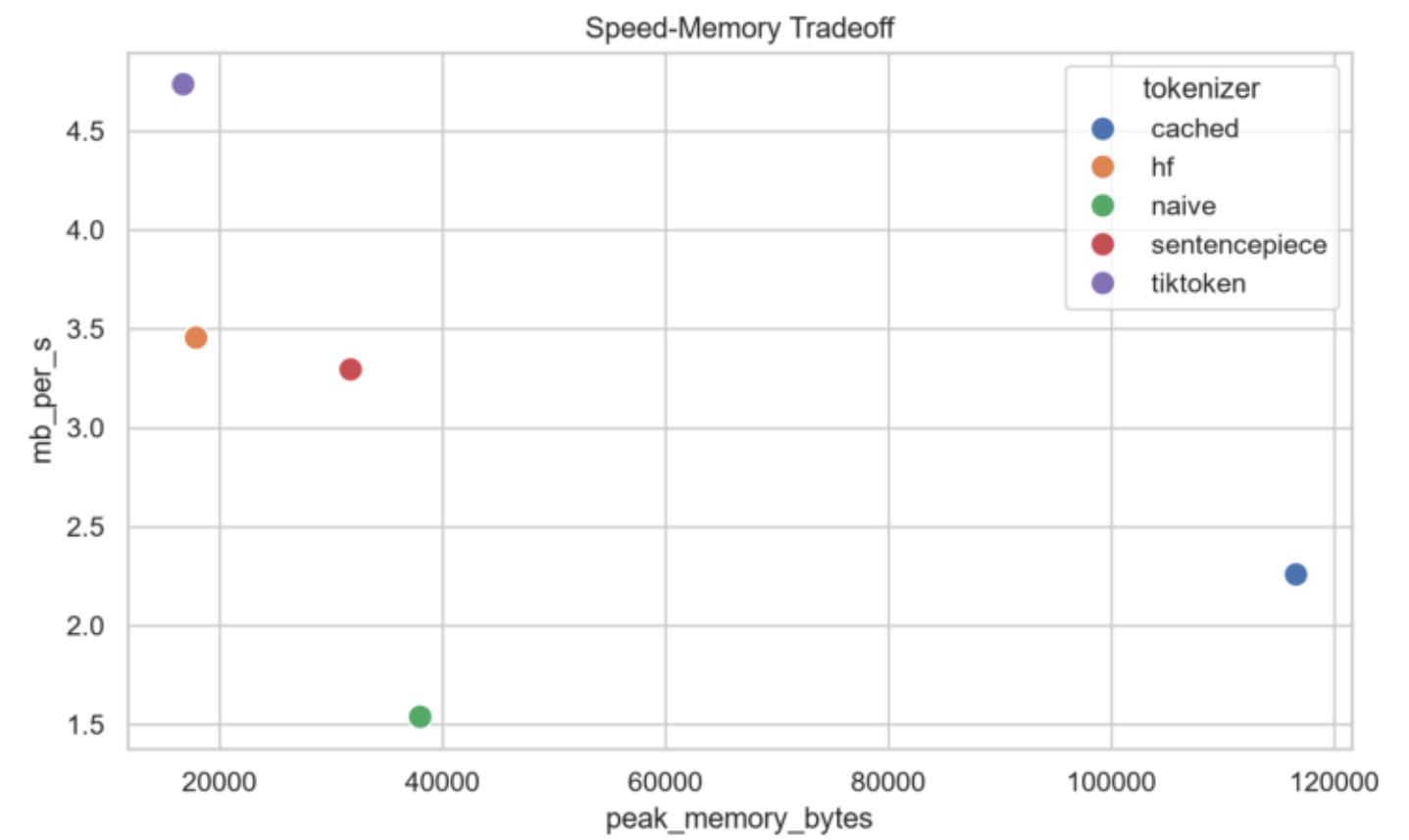
Memory usage by tokenizer and domain.

Cache Hit Rate Vs Speedup



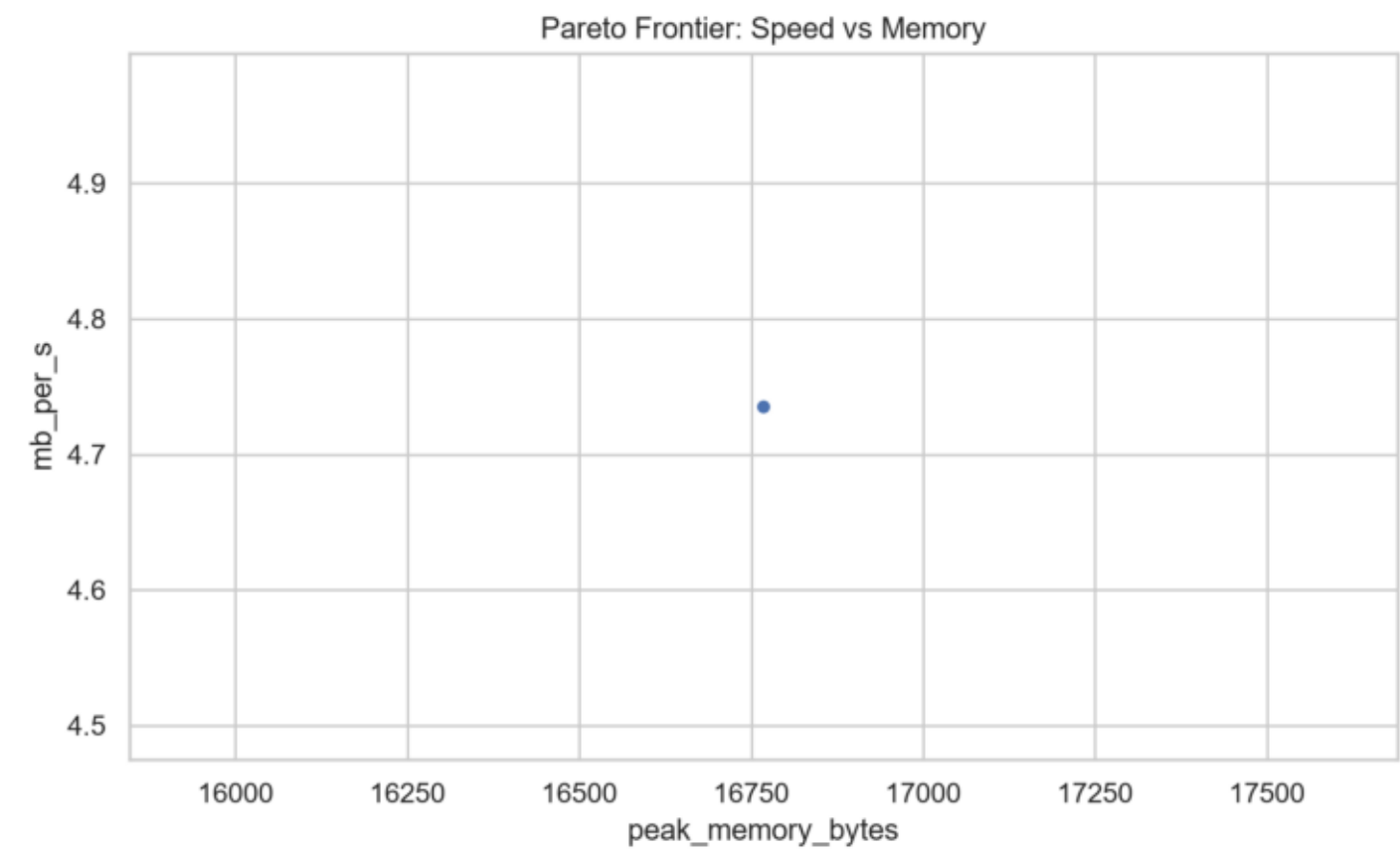
Experiment 6: cache hit rate versus throughput proxy.

Speed Memory Tradeoff



Speed-memory tradeoff across tokenizer implementations.

Pareto Frontier



Pareto frontier of throughput versus memory.